

How to Use Input Files

for dekodeX Problems

Overview

dekodeX problems are designed with realistic constraints, which means input datasets are often too large to type manually. This guide walks you through obtaining the input file for a problem and feeding it into your solution, whether you prefer an online compiler or a local development environment.

Step 1 — Get Your Input File

On the problem page, click the **Get Input** button. You will be taken to a plain-text page containing your personalised test data. Save it with **Ctrl + S** (or copy the entire content) and name the file **input.txt** or **testcase.txt**.

Step 2 — Choose Your Approach

There are two ways to run your solution against the input file:

- **Online Compilers** — paste your code and the input directly into the browser. No local setup required.
- **Local Development** — read the file from disk using standard file I/O in your language of choice.

Online Compilers

Recommended platforms: **OneCompiler** (onecompiler.com/cpp) and **CodeChef IDE** (codechef.com/ide).

Steps: open the platform, paste your code in the editor, then copy the entire contents of your input file and paste it into the **Input** tab. Hit Run.

Tip: Most online compilers expose a dedicated Input panel. If yours does not, look for a Standard Input or stdin field.

Local Development

Place **input.txt** in the same directory as your source file, then use the snippets below to read from it.

C++

solution.cpp

```
#include <iostream>
#include <fstream>
#include <vector>
using namespace std;

int main() {
    ifstream inputFile("input.txt");
    if (!inputFile.is_open()) {
        cerr << "Error: Could not open input file!" << endl;
        return 1;
    }

    int n;
    inputFile >> n;
    vector<int> numbers(n);
    for (int i = 0; i < n; i++) inputFile >> numbers[i];

    inputFile.close();
    // ... your solution logic ...
    return 0;
}
```

Python

solution.py

```
# Method 1 – direct file read
with open('input.txt', 'r') as f:
    lines = f.readlines()
n = int(lines[0].strip())
numbers = list(map(int, lines[1].strip().split()))

# Method 2 – redirect stdin
import sys
sys.stdin = open('input.txt', 'r')
n = int(input())
numbers = list(map(int, input().split()))
# ... your solution logic ...
```

Java

Solution.java

```
import java.io.*;
import java.util.*;

public class Solution {
    public static void main(String[] args) {
        try {
            Scanner scanner = new Scanner(new File("input.txt"));
            int n = scanner.nextInt();
            int[] numbers = new int[n];
            for (int i = 0; i < n; i++) numbers[i] = scanner.nextInt();
            scanner.close();
            // ... your solution logic ...
        } catch (FileNotFoundException e) {
            System.out.println("Error: input.txt not found!");
        }
    }
}
```

Common Issues & Solutions

"File not found" error

Ensure input.txt is in the same directory as your source file. Use **ls** (macOS / Linux) or **dir** (Windows) to list files and confirm the name matches exactly.

Wrong output format

Check for trailing spaces or extra newlines in your print statements. Most problems expect only the answer — no labels, no blank lines.

Runtime errors with large inputs

Use wider data types where needed (e.g., **long long** in C++ for values exceeding 2^{31}). Always check the constraint table in the problem statement before choosing your types.

Online compiler times out

Your algorithm may have a higher time complexity than the problem allows. Review the expected complexity from the constraints and optimise accordingly. If the platform itself is slow, try an alternative such as OneCompiler or CodeChef IDE.

Still Need Help?

If you have followed this guide and are still running into trouble, use the **Contact** option available on the problem page. Include your language, the error message you are seeing, and the steps you have already tried.

Happy Coding.